EAST DELTA UNIVERSITY

CSE 466
PROJECT REPORT

Mental Health Detection Based on Text (i.e.Social Media or Journal Text) Using Python NLP

By
Samiha Jahan Ema
Student ID: 223006612

Date: September 10, 2025

_____

[Samiha Jahan Ema] [223006612]

Edit with WPS Office

ACKNOWLEDGMENTS

I would specially like to thank my instructor, Ms. Arshiyana Shamir, for giving me this opportunity to work on such significant project. It would have been impossible to finish this work without her guidance. I would also like to thank my classmates and seniors for their unwavering supports and assistance at every stage of project. Last but not the least, I would like to thank my family for providing me with the right atmosphere and encouraging me to finish this project.

DECLARATION

I would like to declare that this project work 'Mental Health Detection Based on Text Using Python NLP' is solely my work. The codes and algorithms incorporated have been cited and used as per the academic standard. The work is original and I vouch that the project has been done honorably and with no dishonesty involved.

Edit with WPS Office

TABLE OF CONTENTS

- ACKNOWLEDGMENTS 2
- DECLARATION 2
- ABSTRACT 4
- CHAPTER 1: INTRODUCTION 5
- CHAPTER 2: BACKGROUND 6
- CHAPTER 3: METHODOLOGY 7
- CHAPTER 4: CODE 8
- CHAPTER 5: EXPERIMENTS 12
- CHAPTER 6: RESULTS..... 12
- CHAPTER 7: CONCLUSION 12
- CHAPTER 8: FUTURE WORKS 13
- CHAPTER 9: APPENDIX..... 14

Edit with WPS Office

ABSTRACT

Increasingly, the problems of mental health, such as anxiety, depression, and frustration, have grown, especially with the youth and active users of social media. The methods used for diagnosis are not cost-effective, take a considerable amount of time and carry a stigma. Yet, social media is a rich source of text data that has the potential to indicate mental health problems.

This project uses Natural Language Processing (NLP) and Machine Learning (ML) to identify emotional states from written text. It has been developed through a multi-class classification model that categorizes emotions into five groups: depression, anxiety, frustration, restlessness, and positivity. To train the model, we used TF-IDF feature extraction along with a Naive Bayes classifier.

Users can type texts interactively and see predicted probabilities, results represented as bar charts, and also maintain a session history as an interface designed using `ipywidgets` offers. Although it serves as a proof of concept for larger-scale models, the system performed mental health detection with a moderate degree of success.

Edit with WPS Office

CHAPTER 1: INTRODUCTION

1.1 Problem Statement

The traditional diagnosis process of mental health issues takes time, is convoluted, and is performed by trained professionals. In addition, issues of stigma and access arise. That's why mental health issues like anxiety and depression need automated, scalable, and above all, privacy-focused detection methods at early stages to prevent further complications.

1.2 Problem Solution

The text detection model created by this project aims to identify mental health issues using natural language processing and machine learning. The model is able to classify text based on emotion by using a model that sorts emotional typed responses, and it only requires the user to enter a few sentences to process.

1.3 Objectives

- Prepare a labeled dataset that represents different emotional states.
- Utilize supervised learning techniques for multi-class text classification.
- Create an interactive user interface demo that performs decision on the user inputs.
- Explore the issue of mental health detection based on text using programming in the language of Python.

Edit with WPS Office

CHAPTER 2: BACKGROUND

In recent years, progress in AI and NLP has made it possible to automatically identify psychological traits in written content. Studies have found that people dealing with depression or anxiety tend to use more negative emotion words, refer to themselves in the first person, and convey a sense of hopelessness.

Previous research has had success with TF-IDF vectorization and traditional classifiers like Naive Bayes, SVM, and Logistic Regression, particularly with mental health datasets sourced from platforms such as Reddit or Twitter.

This project takes inspiration from those approaches by creating a small, balanced dataset and training a Naive Bayes classifier to categorize text into five emotional categories. The aim is to show that such systems can work, focusing on the development of a prototype.

Edit with WPS Office

CHAPTER 3: METHODOLOGY

3.1 Dataset Creation

Synonyms and templates were used to increase the textual variety for the generated synthetic dataset containing 120 samples, or 24 per class, which was constructed from the corresponding seed sentences for each class. Text classifiers perform the following tasks sequentially:

3.2 Preprocessing

- Removing extraneous spaces or other meaningless characters from the text.
- Shuffling the text to prevent bias from the order of the text.
- Increasing the number of instances per class to achieve the same number of instances per class for all the classes.

3.3 Model Design

- In the Vectorizer module of TF-IDF, switch each word to a numerical descriptor based on the calculated importance scores assigned, and return values where the features are counted in single and double words, or unigrams and bigrams.
- Bool text classifiers: a model type which, regardless of scale, classifies based solely on the assigned probabilities, which allows for a speed advantage on simpler or smaller datasets.

3.4 Training Process

- Stratified sampling used in the splitting of the dataset, which is divided into an 80% training set and a 20% testing set.
- Scikit-learn pipelines are used to model and build the structures of the classifiers in which the model is then taught.
- Counting the number of predictions to measure classification, recall, f1-score and the number of right predictions to the set maximum which is defined as the accuracy or precision.

3.5 User Interface

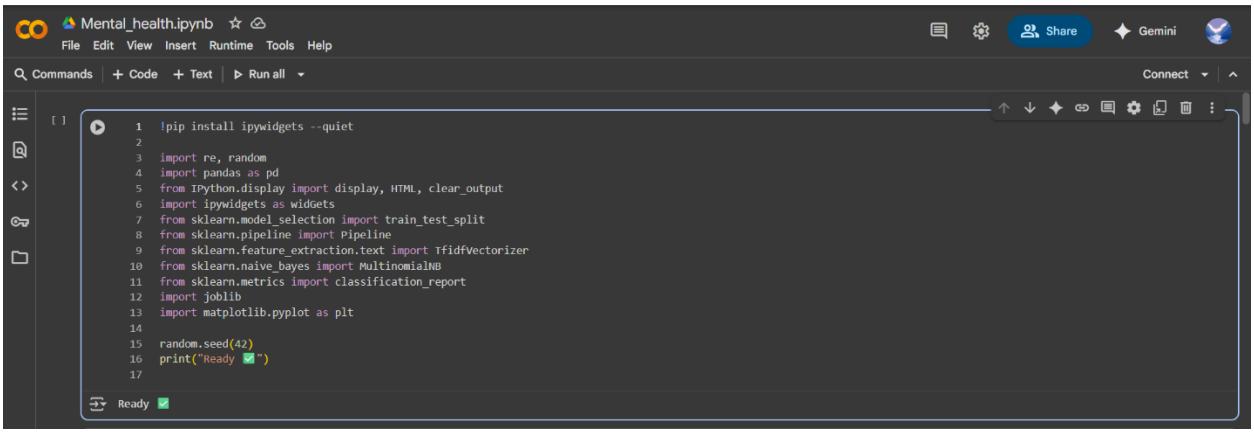
The final presentation layer is done with `ipywidgets` that allows to enter texts, display the output of the analysis in the class regions, display the analysis probabilities in the form of a bar chart, and maintains a session with the last 10 entries on hand.

Edit with WPS Office

CHAPTER 4: CODE

4.1 Requirements

Code Snippet (Cell 1):



Explanation:

The imported libraries needed for this project are: pandas as a book keeping library that allows for easy writing in the form of tables, a set of GUI widgets, tools for as passive and simple text classifiers along with modules for analysis one is the ones that are fronting and thinking the approach with Naive Bayes, and modules for unclenching the myriad of other features with the matplotlibesque modules that are designed for simplified frontends aimed at the users.

4.2 Dataset Generation

Code Snippet (Cell 2):

The suite of tools allows to automatically generate new text info for each of the 5 topics, or classes: depression, anxiety, frustration, restlessness or positiveness. Templates and synonyms are used to create natural-looking variations of text. Here's the output of given cell:

Edit with WPS Office

```
label
depression     24
anxiety        24
frustration    24
restless       24
positive       24
Name: count, dtype: int64
```

	text	label
0	I feel empty and lost	depression
1	Nothing seems worth it anymore	depression
2	I can't find joy in anything	depression
3	Life feels meaningless	depression
4	I feel hopeless	depression
5	I feel like a burden	depression
6	I'm tired of pretending	depression
7	I don't want to get out of bed	depression

Explanation:

- In every batch, 24 auditory samples were provided.
- Textual sentences were randomized and stored in a CSV file (dataMulticlass.csv).
- The dataset maintains relative equality among classes to prevent class imbalance.

4.3 Model Training

Code Snippet (Cell 3):

Mental_health.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect

```python
df = pd.read_csv("dataMulticlass.csv")


XTrain, XTest, yTrain, yTest = train_test_split(
    df["text"], df["label"], test_size=0.2, random_state=42, stratify=df
    ["label"]
)


model = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1,2), min_df=1)),
    ("clf", MultinomialNB())
])

model.fit(XTrain, yTrain)


yPred = model.predict(XTest)
print(classification_report(yTest, yPred, digits=3))

joblib.dump(model, "Mental_Health_Multiclass.pkl")

print("Model trained and saved ✅")
```

```
              precision    recall  f1-score   support

     anxiety      0.429     0.600     0.500         5
  depression      0.667     0.500     0.571         4
frustration       0.600     0.600     0.600         5
    positive      0.333     0.200     0.250         5
    restless      0.500     0.600     0.545         5

    accuracy                          0.500        24
   macro avg      0.506     0.500     0.493        24
weighted avg      0.499     0.500     0.490        24

Model trained and saved ✅
```


Edit with WPS Office

Explanation:

- The dataset is split into a training set (80%) and a testing set (20%).
- A pipeline consisting of TF-IDF + a multivariate Naive Bayes classifier is constructed.
- The model is trained and tested on the provided models' testing set.
- The observed metrics of the classification indicate a classification accuracy of 50%.
- The model that the user provided is stored in the file Mental_Health_Multiclass.pkl.

4.4 Prediction Functions

Code Snippet (Cell 4):

```python
model = joblib.load("Mental_Health_Multiclass.pkl")

def predictPercentages(text: str):
    proba = model.predict_proba([text])[0]
    classes = list(model.classes_)
    return {cls: round(float(p)*100, 2) for cls, p in zip(classes, proba)}

def topLabel(probs: dict):
    return max(probs.items(), key=lambda kv: kv[1])

def colorFor(label: str):
    return {
        "depression": "#cc0000",
        "anxiety": "#e67e22",
        "frustration": "#c0392b",
        "restless": "#8e44ad",
        "positive": "#2e7d32",
    }.get(label, "#333333")

print("Ready to predict ✅")
```

```
Ready to predict ✅
```

Functions are provided to:

- Determine the probabilities associated with a prediction.
- Ascertain the plurality class.
- Set meaningful colors for each class.

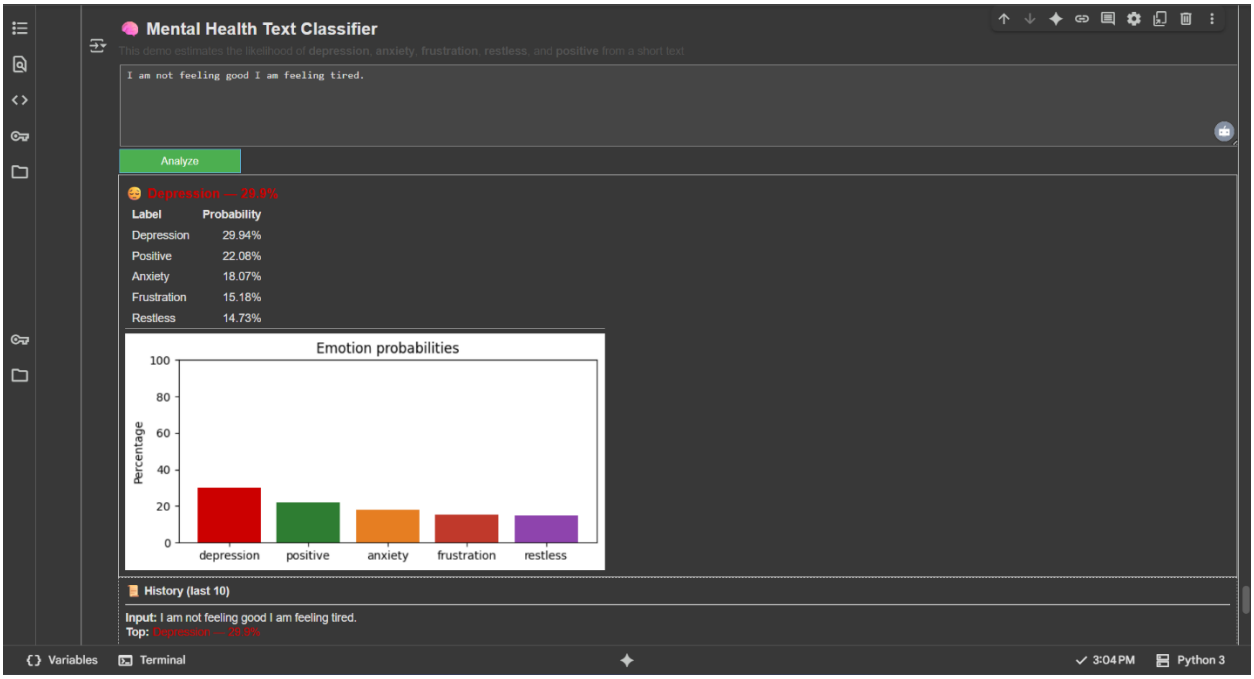
Explanation:

These classes behavior in such a way as to offshore the model outputs to the user and expose the outputs as pie charts and colors.

4.5 User Interface

Edit with WPS Office

Code Snippet (Cell 5):
Implements input box, analyze button and output display with history.



Explanation:

- A user writes a short description of the emotional state they are going through.
- The model calculates the probability for each category.
- The model summarizes the results in a text format with emojis, a probability table, and a bar chart.
- The model also records the 10 most recent user queries and results for a complete session.

Edit with WPS Office

CHAPTER 5: EXPERIMENTS

Experiments were done on a synthetized dataset with a total of 120 samples.

- **Training:** 96 samples for training.
- **Testing:** 24 samples for testing.
- **Classifier**: Multinomial Naive Bayes.

Observed Metrics:

- Depression (Top score, F1 = 0.57).
- Positive (Imperfect, F1 = 0.25).
- Overall Accuracy (Approximately 50%)

CHAPTER 6: RESULTS

The project highlighted:

- Formulation of a balanced multi-class dataset batches.
- Training of a Native Bayes classifier with TF-IDF features.
- Implementation of a classifier UI for interective text classifier
- Conceptual representation of mental health state classification from text.

Edit with WPS Office

CHAPTER 7: CONCLUTIONS

The representation provided is for a working system, but it is limited by dataset size.

Segmentation of text for mental health concern is evaluated using Python NLP and ML in this project. A synthetic dataset was developed, and a remote user interface was built for real time Naive Bayes emotion classification into five groups. A system trained to classify mental health texts and a user interface for real time classification were developed.

The project is proof of concept, but also, the small sets and simplistic models simultaneously represent the foundation for future studies in automation to detect mental health.

CHAPTER 8: FUTURE WORKS

1. More real-world complex datasets - Annotated collections of social media and personal journals are needed.

2. More sophisticated methods - Application of deep learning techniques such as LSTM and BERT to the problem.

3. Better Augmentation - Paraphrasing with Transfer Learning is needed to create a larger dataset.

4. Implementation - Extend the solution into a practical web or mobile application.

5. Ethics & Privacy - Ensure there is consent and anonymity to responsibly use the predictions made.

Edit with WPS Office

CHAPTER 9: APPENDIX

cell 1

```
!pip install ipywidgets --quiet


import re, random

import pandas as pd

from IPython.display import display, HTML, clear_output

import ipywidgets as widGets

from sklearn.model_selection import train_test_split

from sklearn.pipeline import Pipeline

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import classification_report
```


Edit with WPS Office

```
import joblib

import matplotlib.pyplot as plt


random.seed(42)

print("Ready ✅")


Cell 2

seed = {

    "depression": [

        "I feel empty and lost", "Nothing seems worth it anymore", "I can't find joy in anything",

        "Life feels meaningless", "I feel hopeless", "I feel like a burden", "I'm tired of pretending",

        "I don't want to get out of bed", "I keep crying without reason", "I feel numb most days"

    ],

    "anxiety": [

        "I can't stop worrying", "My heart races and I panic", "I feel nervous about everything",

        "I'm afraid of making mistakes", "I overthink all the time", "I feel on edge",

        "I dread tomorrow", "I'm scared about the future", "My thoughts spiral", "I fear losing control"

    ],

    "frustration": [

        "I'm so frustrated with myself", "Everything is annoying today", "I keep failing and it hurts",

        "I'm fed up with this situation", "Nothing goes my way", "I feel angry and stuck",
```


Edit with WPS Office

```
        "I can’t stand repeating the same errors", "This is getting on my nerves",
        "I feel irritated at everyone", "Why won't things just work?"
    ],
    "restless": [
        "I can't relax at all", "I keep pacing around the room", "I feel jittery and unsettled",
        "My mind won’t slow down", "I can’t sit still", "I feel fidgety",
        "I keep tossing and turning all night", "I feel wired but tired",
        "I can’t focus on one thing", "I feel constantly unsettled"
    ],
    "positive": [
        "I feel calm and grateful", "I had a wonderful day", "Everything is falling into place",
        "I'm proud of my progress", "I feel hopeful and strong", "I'm happy with my life",
        "I enjoyed time with family", "I’m excited for tomorrow", "I feel confident",
        "I’m at peace with where I am"
    ]
}


templates = [
    "Today I {verb} {feeling}.",
    "Lately, I {verb} {feeling}.",
    "Sometimes I {verb} {feeling}, and it’s hard.",
```


Edit with WPS Office

```
    "I just {verb} {feeling}.",
    "Right now, I {verb} {feeling}.",
]

verbalizers = {
    "depression": ["feel", "am feeling", "have been feeling"],
    "anxiety": ["feel", "am feeling", "keep feeling"],
    "frustration": ["feel", "am feeling", "keep getting"],
    "restless": ["feel", "am feeling", "keep feeling"],
    "positive": ["feel", "am feeling", "have been feeling"],
}

synonyms = {
    "depression": ["empty", "lost", "down", "hopeless", "numb", "low", "meaningless"],
    "anxiety": ["anxious", "worried", "uneasy", "panicky", "nervous", "on edge"],
    "frustration": ["frustrated", "irritated", "annoyed", "fed up", "angry", "stuck"],
    "restless": ["restless", "jittery", "unsettled", "fidgety", "wired", "unable to relax"],
    "positive": ["grateful", "hopeful", "calm", "happy", "content", "optimistic", "peaceful"]
}

def cleanSpace(s: str) -> str:
    return re.sub(r"\s+", " ", s).strip()
```


Edit with WPS Office

```
def generateClassSamples(label, n_target=24):
    base_li = seed[label][:]

    out_li = base_li[:]

    while len(out_li) < n_target:
        tmpl = random.choice(templates)
        verb = random.choice(verbalizers[label])
        feel = random.choice(synonyms[label])
        sent = tmpl.format(verb=verb, feeling=feel)

        extra = [
            "",
            "and I don't know what to do",
            "but I’m trying to cope",
            "and I wish it would stop",
            "though I’m managing better today",
            "and it keeps coming back",
            "but I’m taking it one step at a time"
        ]
        sent = cleanSpace(f"{sent} {random.choice(extra)}.")
        out_li.append(sent)
    return out_li[:n_target]
```


Edit with WPS Office

```
perClass = 24
allTexts, allLabels = [], []

for lbl in ["depression", "anxiety", "frustration", "restless", "positive"]:
    samples = generateClassSamples(lbl, n_target=per_class)
    allTexts.extend(samples)
    allLabels.extend([lbl] * len(samples))


a = list(zip(allTexts, allLabels))
random.shuffle(a)
all_texts, all_labels = zip(*a)

df = pd.DataFrame({"text": allTexts, "label": allLabels})
df.to_csv("dataMulticlass.csv", index=False)

print(df['label'].value_counts())
df.head(8)

cell 3
df = pd.read_csv("dataMulticlass.csv")
```


Edit with WPS Office

```
XTrain, XTest, yTrain, yTest = train_test_split(
    df["text"], df["label"], test_size=0.2, random_state=42, stratify=df["label"]
)
model = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1,2), min_df=1)),
    ("clf", MultinomialNB())
])
model.fit(XTrain, yTrain)
yPred = model.predict(XTest)
print(classification_report(yTest, yPred, digits=3))

joblib.dump(model, "Mental_Health_Multiclass.pkl")
print("Model trained and saved ✅")
```

Model trained and saved ✅

cell 4

```
model = joblib.load("Mental_Health_Multiclass.pkl")
def predictPercentages(text: str):
    proba = model.predict_proba([text])[0]
    classes = list(model.classes_)
    return {cls: round(float(p)*100, 2) for cls, p in zip(classes, proba)}

def topLabel(probs: dict):
```


Edit with WPS Office

```
    return max(probs.items(), key=lambda kv: kv[1])
def colorFor(label: str):
    return {
        "depression": "#cc0000",
        "anxiety": "#e67e22",
        "frustration": "#c0392b",
        "restless": "#8e44ad",
        "positive": "#2e7d32",
    }.get(label, "#333333")


print("Ready to predict ✅")


cell 5
inp = widGets.Textarea(
    value='',
    place_holder="Please type how you're feeling __ (e.g., 'I'm not feeling good and I feel frustrated about everything today')",
    des_cription='Input:',
    layout=widGets.Layout(width='100%', height='100px')
)


btn = widGets.Button(description='Analyze', button_style='success')
out = widGets.Output(layout={'border': '1px solid #ddd', 'padding': '8px'})
hist = widGets.Output(layout={'border': '1px dashed #bbb', 'padding': '8px'})
```


Edit with WPS Office

```
historyItems = []

def drawBar(probs: dict):

    items = sorted(probs.items(), key=lambda kv: kv[1], reverse=True)
    labels = [k for k,_ in items]
    values = [v for _,v in items]
    colors = [colorFor(k) for k in labels]
    plt.figure(figsize=(6,3))
    plt.bar(labels, values, color=colors)
    plt.ylim(0, 100)
    plt.ylabel("Percentage")
    plt.title("Emotion probabilities")
    plt.tight_layout()
    plt.show()

def renderHistory():
    with hist:
        clear_output()
        display(HTML("<h4 style='margin:0'>📜 History (last 10)</h4><hr/>"))
        for item in historyItems[-10:][::-1]:
            label, pct = item['top']
```


Edit with WPS Office

```
        color = colorFor(label)
        display(HTML(
            f"<div style='margin-bottom:8px'>"
            f"<div><b>Input:</b> {item['text']}</div>"
            f"<div><b>Top:</b> <span style='color:{color}'>{label.title()} — {pct:.1f}%</span></div>"
            f"</div><hr/>"
        ))


def onClick(_):
    with out:
        clear_output()
        text = inp.value.strip()
        if not text:
            display(HTML("<span style='color:#777'>Please enter some text.</span>"))
            return
        probs = predictPercentages(text)
        label, pct = topLabel(probs)
        # Emoji and headline color
        emoji = {"positive":"🙂","anxiety":"😰","frustration":"😠","depression":"😔","restless":"😵‍💫"}.get(label, "🧠")
        color = colorFor(label)
        display(HTML(
            f"<h3 style='margin:4px 0;color:{color}'>{emoji} {label.title()} — {pct:.1f}%</h3>"        ))
```


Edit with WPS Office

```
    rows = "".join(
        f"<tr><td style='padding:4px 8px'>{k.title()}</td>"
        f"<td style='padding:4px 8px;text-align:right'>{v:.2f}%</td></tr>"
        for k,v in sorted(probs.items(), key=lambda kv: kv[1], reverse=True)
    )
    display(HTML(
        "<table style='border-collapse:collapse'>"
        "<thead><tr><th style='text-align:left;padding:4px 8px'>Label</th>"
        "<th style='text-align:right;padding:4px 8px'>Probability</th></tr></thead>"
        f"<tbody>{rows}</tbody></table>"
    ))

    drawBar(probs)

    historyItems.append({"text": text, "probs": probs, "top": (label, pct)})
    renderHistory()

btn.on_click(onClick)
display(HTML("<h2 style='margin-bottom:6px'>🧠Mental Health Text Classifier</h2>"
         "<p style='margin-top:0;color:#555'>This demo estimates the likelihood of "
         "<b>depression</b>, <b>anxiety</b>, <b>frustration</b>, <b>restless</b>, and <b>positive</b> "
         "from a short text</p>"))
```


Edit with WPS Office

```
UI = widGets.VBox([inp, btn, out, hist])

display(UI)
```


Edit with WPS Office

Edit with WPS Office